

● Gate-Level Modeling



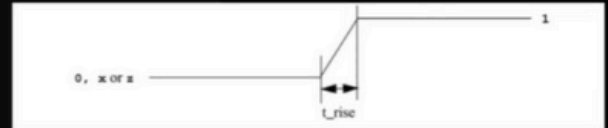
Gate Delays

Gate delays in Verilog allow designers to model the timing behavior of circuits, specifying the time taken for signal transitions.

Rise, Fall, and Turn-off Delays

1. Types of Delays:

- **Rise Delay:** Transition to logic 1 from any other value (0, x, or z).
- **Fall Delay:** Transition to logic 0 from any other value (1, x, or z).
- **Turn-off Delay:** Transition to high impedance (z) from any other value.



● Verilog Framework

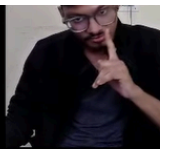


2. Delay Specifications:

- **Single Delay:** Applied to all transitions (rise, fall, and turn-off).
- **Two Delays:** Specifies separate rise and fall delays. Turn-off delay is the minimum of these two.
- **Three Delays:** Specifies rise, fall, and turn-off delays independently.

Examples:

- Single delay: `and #5 a1(out, i1, i2);` (5 for all transitions).
- Two delays: `and #4,6 a2(out, i1, i2);` (rise = 4, fall = 6).
- Three delays: `bufif0 #3,4,5 b1(out, in, control);` (rise = 3, fall = 4, turn-off = 5).



Min/Typ/Max Values

1. **Purpose:** Used to model the variability in gate delays caused by IC fabrication process differences. Designers can specify:

- **Min Value:** Minimum expected delay.
- **Typ Value:** Typical (default) delay value.
- **Max Value:** Maximum expected delay.

2. **Runtime Control:**

- Select min, typ, or max at runtime using simulator options (+mindelays, +typdelays, or +maxdelays).
- Default is the **typ** delay if no option is specified.

```
// One delay
// if +mindelays, delay= 4
// if +typdelays, delay= 5
// if +maxdelays, delay= 6
and #(4:5:6) a1(out, i1, i2);

// Two delays
// if +mindelays, rise= 3, fall= 5, turn-off = min(3,5)
// if +typdelays, rise= 4, fall= 6, turn-off = min(4,6)
// if +maxdelays, rise= 5, fall= 7, turn-off = min(5,7)
and #(3:4:5, 5:6:7) a2(out, i1, i2);

// Three delays
// if +mindelays, rise= 2 fall= 3 turn-off = 4
// if +typdelays, rise= 3 fall= 4 turn-off = 5
// if +maxdelays, rise= 4 fall= 5 turn-off = 6
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1, i2);
```

Examples of invoking the Verilog-XL simulator with the command-line options are shown below. Assume that the module with delays is declared in the file test.v.

```
//invoke simulation with maximum delay
> verilog test.v +maxdelays
```

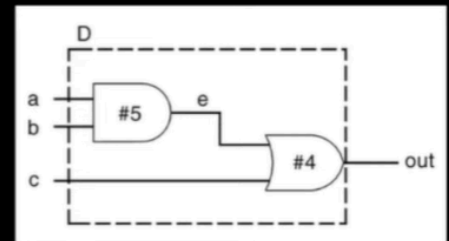
```
//invoke simulation with minimum delay
> verilog test.v +mindelays
```

```
//invoke simulation with typical delay
> verilog test.v +typdelays
```

Delay Example

1. **Module Description:**

- A simple combinational circuit is defined with the logic equation:
 $out = (a \& b) + c$.
- The circuit consists of two gates:
 - AND gate with delay = 5 units.
 - OR gate with delay = 4 units.



Verilog Definition:

```
Verilog

module D (out, a, b, c);
    output out;
    input a, b, c;
    wire e;
    and #5 a1(e, a, b); // Delay of 5
    or #4 o1(out, e, c); // Delay of 4
endmodule
```

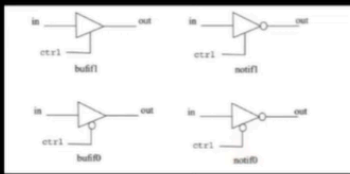
Verilog

```
module stimulus;
    reg A, B, C;
    wire OUT;
    D d1 (OUT, A, B, C);

    initial begin
        A = 1'b0; B = 1'b0; C = 1'b0;
        #10 A = 1'b1; B = 1'b1; C = 1'b1;
        #10 A = 1'b1; B = 1'b0; C = 1'b0;
        #20 $finish;
    end
endmodule
```

➤ BUF/NOT Gates

BUF and NOT gates can also include control signals (bufif1, bufif0, notif1, notif0) to propagate signals conditionally.



bufif1	ctrl				in	out
	0	1	x	z		
0	z	0	L	L		
1	z	1	H	H		
x	z	x	x	x		
z	z	x	x	x		

bufif0	ctrl				in	out
	0	1	x	z		
0	0	z	L	L		
1	1	z	H	H		
x	x	z	x	x		
z	x	z	x	x		

notif1	ctrl				in	out
	0	1	x	z		
0	z	1	H	H		
1	z	0	L	L		
x	z	x	x	x		
z	z	x	x	x		

notif0	ctrl				in	out
	0	1	x	z		
0	1	z	H	H		
1	0	z	L	L		
x	x	z	x	x		
z	x	z	x	x		

```
module conditional_buf;

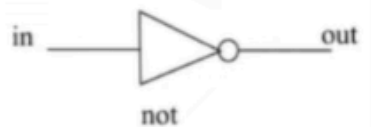
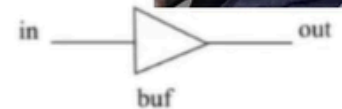
wire OUT1,OUT2, IN, CTRL;
assign IN = 1; //Play around with the values
assign CTRL = 0;
// Conditional buffer
bufif1 buf1(OUT1, IN, CTRL); // Propagates input only if CTRL is 1
notif0 not1(OUT2, IN, CTRL); // Produces inverted output if CTRL is 0

initial #5 $display("Out1 : %0b, Out2 : %0b",OUT1,OUT2);
initial #30 $finish;

endmodule
```

buf	in	out
	0	0
	1	1
	x	x
	z	x

not	in	out
	0	1
	1	0
	x	x
	z	x



```
module buf_not_gate_instantiations;

wire OUT1, OUT2,OUT3,OUT4, IN;
assign IN = 1; //Play around with the values

// Single-output gates
buf buf1(OUT1, IN); // Buffer
not not1(OUT2, IN); // Inverter

// Multiple-output buffer
buf buf2(OUT3, OUT4, IN);

initial #5 $display("Out1 : %0b, Out2 : %0b,Out3 : %0b,Out4 : %0b",OUT1,OUT2,OUT3,OUT4);
initial #30 $finish;

endmodule
```

		il			
and		0	1	x	z
i2	0	0	0	0	0
	1	0	1	x	x
	x	0	x	x	x
	z	0	x	x	x

		il			
nand		0	1	x	z
i2	0	1	1	1	1
	1	1	0	x	x
	x	1	x	x	x
	z	1	x	x	x

		il			
or		0	1	x	z
i2	0	0	1	x	x
	1	1	1	1	1
	x	x	1	x	x
	z	x	1	x	x

		il			
nor		0	1	x	z
i2	0	1	0	x	x
	1	0	0	0	0
	x	x	0	x	x
	z	x	0	x	x

		il			
xor		0	1	x	z
i2	0	0	1	x	x
	1	1	0	x	x
	x	x	x	x	x
	z	x	x	x	x

		il			
xnor		0	1	x	z
i2	0	1	0	x	x
	1	0	1	x	x
	x	x	x	x	x
	z	x	x	x	x

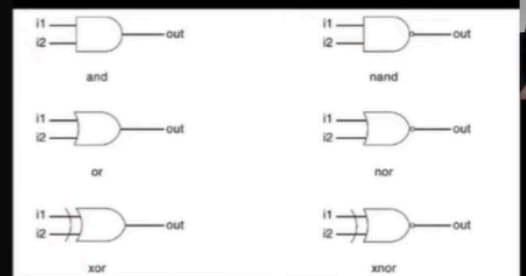
Gate-Level Modeling



Logic Symbols:

The corresponding symbols for these gates are shown below:

- **AND Gate:** Output is high when all inputs are high.
- **OR Gate:** Output is high when at least one input is high.
- **XOR Gate:** Output is high when inputs differ.
- **NAND/NOR/XNOR:** Complemented versions of the above gates.



Gate Instantiations:

Gates are instantiated by specifying their type, output, and input(s). Instance names are optional, allowing designers to instantiate multiple gates without naming each explicitly. For gates with more than two inputs, additional ports can be added.

```
module gate_instantiations;
    wire OUT1,OUT2,OUT3, IN1, IN2, IN3;
    assign IN1 = 1; //Play around with this values
    assign IN2 = 0;
    assign IN3 = 1;

    // Two-input gates
    and and1(OUT1, IN1, IN2); // AND Gate
    nand nand1(OUT2, IN1, IN2); // NAND Gate

    // More than two inputs
    nand nand3(OUT3, IN1, IN2, IN3); // 3-input NAND Gate

    initial #5 $display("Out1 : %b, Out2 : %b, Out3 : %b",OUT1,OUT2,OUT3);
    initial #30 $finish;
endmodule
```